

UNIT – I

BASIC STRUCTURES OF COMPUTER: Functional Units, Multiprocessors and Multicomputers, Memory Locations and Addresses, Memory operations, Instructions and Instruction Sequencing, Addressing modes, Assembly Language, Basic Input/Output operations, Stacks and Queues, Subroutines, Shift and rotate Instructions, Byte-Sorting program.

2 MARKS

1. What are the functional units?

A computer consists of five functionally independent main parts. They are

1. Input
2. Memory
3. arithmetic and logic
4. output and control units

2. What is meant by input unit?

Computers accept coded information through input units, which read the data. The most well-known input device is the keyboard. Whenever a key is pressed, the corresponding letter or digit is automatically translated into its corresponding binary code and transmitted over a cable to either the memory or the processor

3. What is meant by memory unit?

The function of the memory unit is to store programs and data. There are two classes of storage, called primary and secondary.

Primary storage is a fast memory that operates at electronic speeds. Programs must be stored in the memory while they are being executed.

Memory in which any location can be reached in a short and a fixed amount of time after specifying its address is called random access memory (RAM) the time required to access one word is called memory access time.

4. What are the operations in ALU?

Arithmetic and logic operations – Multiplication, division, comparison of numbers etc.

When operands are brought into the processor they are stored in high speed storage elements called Registers. Each Register can store one word of data

5. What is meant by output unit?

The output unit is the counter part of the input unit. Its function is to send processed result to the outside world. Output units – Printer (inkjet printer, Laser printer). Graphics display provide both an input function and output function-I/O Unit.

6. What are the operations in control unit?

The operation of a computer can be summarized as follows,

1. the computer accepts information in the form of programs and data through an input unit and stores it in the memory
2. information stored in the memory is fetched, under program control, into an arithmetic and logic unit, where it is processed.
3. Processed information leaves the computer through an output unit.
4. All activities inside the machines are directed by the control unit.

7. Define byte addressability?

- Three basic information – bit, byte and word
- A byte is always 8 bits. But the word length typically ranges from 16 to 64 bits.
- Memory location assignments refer to successive byte locations in memory
- Memory is byte-accessible.
- For 8086, a Word is 16-bits (2 bytes)

8. What is meant by multiprocessor?

Large computer system may contain a number of processor units, in which case they are called multiprocessor systems.

These systems either execute a number of different application tasks in parallel, or they execute sub-tasks of a single large task in parallel.

9. What is meant by multicomputers?

The high performance of these systems comes with much increased complexity and cost in contrast to multiprocessor systems, it is also possible to use an interconnected group of complete computers to achieve high total computational power.

When the tasks they are executing need to communicate data, they do so by exchanging messages over a communication network.

This property distinguishes them from shared memory multiprocessors, leading to the name message passing multicomputer.

10. What is meant by Big-Endian?

Big-Endian – lower byte addresses are used for the more significant bytes of a word.

Word address	Byte address			
0	0	1	2	3
4	4	5	6	7
2^k-4	2^k-4	2^k-3	2^k-2	2^k-1

(a) Big-endian assignment

11. What is meant by little endian?

Little-Endian – lower byte addresses are used for the less significant bytes of a word

Byte address				
0	3	2	1	0
4	7	6	5	4
2^k-4	2^k-1	2^k-2	2^k-3	2^k-4

(b) Little-endian assignment

12. How to align the word?

In the case of 32-bit word length, natural word boundaries occur at addresses 0, 4, 8 as shown in the (fig. byte and word addressing). The word locations have aligned addresses. Words are said to be aligned in memory if they begin at a byte address that is a multiple of the number of bytes in a word.

There is no fundamental reason why words cannot begin at an arbitrary byte address. In that case words are said to have unaligned addresses.

13. What are the Ways to indicate the length of the string?

There are two ways to indicate the length of the string.

1. The special control character with the meaning “end of string” can be used as last character in the string.
2. Processor register can contain a number indicating the length of the string in bytes.

14. What are the memory operations?

- To execute an instruction, the processor control circuits must cause the word containing the instruction to be transferred from the memory to the processor
- Two basic operations involving the memory are needed, namely load and store.
- load operation
- store operation

15. Define subroutine?

Subroutine calls are a special type of branch where we return to one instruction below the calling instruction.

Provision must be made to save the return address, since it cannot be written into ROM.

16. What are the basic I/O operations?

- Program controlled IO
- Memory mapped IO

17. Explain stack?

A stack is a list of data elements, usually words or bits, with the accessing restrictions that elements can be added or removed at one end of the list only. This end is called the top of the stack and other end is called the bottom. The structure is sometimes referred to as Push down stack. It uses LIFO principle for two operations-Push and Pop.

18. Explain Queue?

Another useful datastructure that is similar to stack is called the queue. Data are stored and retrieved from a queue on FIFO basics. New data are added at the back and retrieved from the front..

19. What is the difference between stack and queue implementation?

There are 2 important differences between how a stack and a queue is implemented.

One end of the is fixed while the other end rises and falls as data are pushed and popped. A single pointer is needed to point to top of the stack, at on the other hand both ends of the queue move to higher addresses as data are added at the back and removed from the front.

So two pointers are needed to keep track of the two ends of the queue.

One way to limit the queue to fixed region in a memory is to use a circular buffer.

20. Define Assembly language?

Machine Instructions Are Represented By patterns of 0s and 1s. symbolic names are used to represent patterns. When writing programs for specific computer, such words are normally replaced by acronyms called mnemonics such as MOV, ADD, INC, and BR. R3 is used to refer register 3, and LOC to refer to a location.

A complete set of such symbolic names and rules for their use constitute a programming language, generally referred to as an assembly language.

21. What are the basic operations performed by any computer system?(Nov 2012)

Basic operations of a computer

Inputting

The process of entering the data and instruction in the computer system.2.

Processing

Performing arithmetic operations or logical on data to convert them into useful information3.

Outputting

The process of producing the useful information or result for the user such as a printed report or visual display4.

Storing

Storing data and instruction to make them readily available for initial or additional processing whenever required.5.

Controlling

Directing the manner or sequence in which all of the above operation are performed

22. What is syntax of the language?

The set of rules for using the mnemonics in the specification of complete instructions and programs is called the syntax of the language.

23. What are the instructions does the instructions consists of?

A typical assembly language consists of 3 types of instruction statements that are used to define program operations:

- Opcode mnemonics
- Data definitions
- Assembly directives

24. List the various phases of an instruction cycle?(Nov 2012)

Fetch, Decode & Execute.

25. Define Subroutine Nesting.

Subroutine nesting is to have one subroutine call another. In this case, the return address of the second call is also stored in the link register, destroying its previous contents. Hence, it is essential to save the contents of the link register in some other location before calling another subroutines. Otherwise, the return address of the first subroutine will be lost.

26. Define The Processor Stack.

The stack pointer points to the stack called the processor stack. The call instruction pushes the contents of the PC on to the processor stack and loads the subroutine address into the PC. The return instruction pops the return address from the processor stack into the PC.

27. Define shift and rotate instructions?

There are many applications that require the bits of an operand to be shifted right or left some specified number of bit positions.

The details of how the shifts are performed depend on whether the operand is a signed number or some more general binary coded information. For general operand we use a logical shift, for number we use an arithmetic shift which preserves the sign of the number.

Logical Shift:

Two logical shift instructions are needed, one for shifting left (LShiftL) and another for shifting right (LShiftR).

28. What is meant by subroutine nesting?

Subroutine nesting is to have one subroutine call another. In this case, the return address of the second call is also stored in the link register, destroying its previous contents.

Hence, it is essential to save the contents of the link register in some other location before calling another subroutines. Otherwise, the return address of the first subroutine will be lost.

FUNCTIONAL UNITS

A computer consists of five functionally independent main parts. They are

5. Input
6. Memory
7. arithmetic and logic
8. output and
9. control units

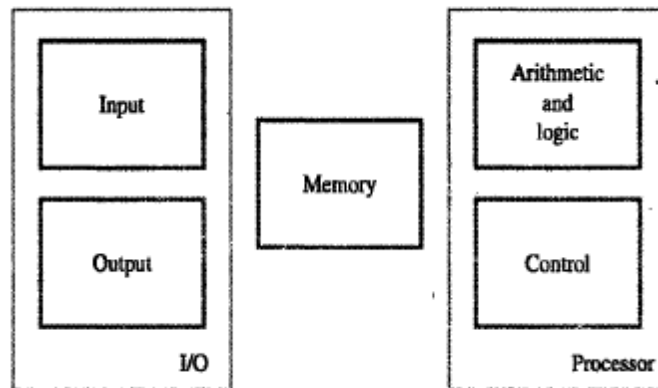


Fig.1.1 Basic functional units of a computer

INPUT UNIT

Computers accept coded information through input units, which read the data. The most well-known input device is the keyboard. Whenever a key is pressed, the corresponding letter or digit is automatically translated into its corresponding binary code and transmitted over a cable to either the memory or the processor

MEMORY UNIT

The function of the memory unit is to store programs and data. There are two classes of storage, called primary and secondary.

Primary storage is a fast memory that operates at electronic speeds. Programs must be stored in the memory while they are being executed. The memory contains the large number of semi-conductor storage cells, each capable of storing one bit of information. These cells are rarely read or written as individual cells but instead are processed in groups of fixed size called words.

To provide easy access to any word in the memory, a distinct address is associated with each word location. Addresses are numbers that identify successive locations. The number of bits in each word is often referred to as the word length of the computer. Typical word length ranges from 16 to 64 bits.

Memory in which any location can be reached in a short and a fixed amount of time after specifying its address is called random access memory (RAM). The time required to access one word is called memory access time.

The memory of the computer is normally implemented as a memory hierarchy of three or four levels of semi-conductor RAM units with different speeds and sizes. The small, fast RAM units are called Caches. The largest and slowest units are referred to as the main memory.

Primary storage is expensive so in addition secondary storage is used when large amount of data and many programs have to be stored, particularly for information i.e. accessed infrequently. Secondary storage is also cheaper. The wide selection of secondary storage device is available, including magnetic disc and tapes and optical disc, CD-ROM.

ARITHMETIC AND LOGIC UNIT

Most computer operations are executed in the arithmetic and logic units (ALU) of the processor.

Arithmetic and logic operations – Multiplication, division, comparison of numbers etc.,

When operands are brought into the processor they are stored in high speed storage elements called Registers. Each Register can store one word of data

OUTPUT UNIT

The output unit is the counter part of the input unit. Its function is to send processed result to the outside world. Output units – Printer (inkjet printer, Laser printer). Graphics display provide both an input function and input function-I/O Unit.

CONTROL UNIT

The control unit is effectively the node center that sends control signals to other units and senses their states. The actual timing signals that governs the transfer or generated by the control circuit. The operation of a computer can be summarized as follows,

5. the computer accepts information in the form of programs and data through an input unit and stores it in the memory
6. information stored in the memory is fetched, under program control, into an arithmetic and logic unit, where it is processed.
7. Processed information leaves the computer through an output unit.
8. All activities inside the machines are directed by the control unit.

MULTIPROCESSOR AND MULTICOMPUTERS
--

Large computer system may contain a number of processor units, in which case they are called multiprocessor systems. These systems either execute a number of different application task in parallel, or they execute sub task of a single large in parallel. All processors usually have access to all of the memory in such systems, and the term shared memory multiprocessor systems is often is used to make this clear.

The high performance of these systems comes with much increased complexity and cost in contrast to multiprocessor systems, it is also possible to use an interconnected group of complete computer to achieve high total computational power. when the task they are executing need to communicate data, they do so by exchanging messages over a communication network.

This property distinguishes them from shared memory multiprocessors, reading to the name message passing multicomputer.

MEMORY LOCATIONS AND ADDRESSES

Number and character operands, as well as instructions, are stored in the memory of the computer. The memory consists of many millions of storage cells each of which can store a bit of information having the values zero or one. Each group of n bits is refer to as a word of information, and n is called the word length.

Accessing the memory to store or retrieve a single item of information, either a word or a byte, requires distinct names or addresses for each item location. The addresses of successive location in the memory can have values from 0 to $2^k - 1$.

BYTE ADDRESSBILITY

- Three basic information – bit, byte and word
- A byte Is always 8 bits. But the word length typically ranges from 16 to 64 bits.
- Memory location assignments refer to successive **byte** locations in memory
- Memory is byte-accessible.
- For 8086, a Word is 16-bits (2 bytes)

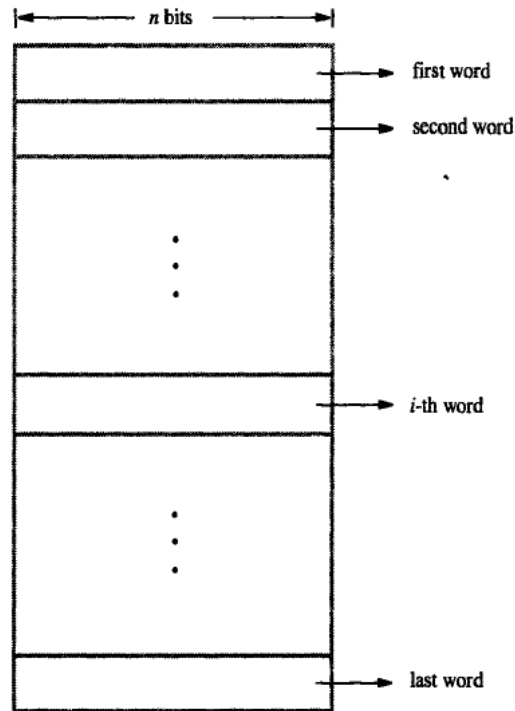


Fig.1.2 Memory words

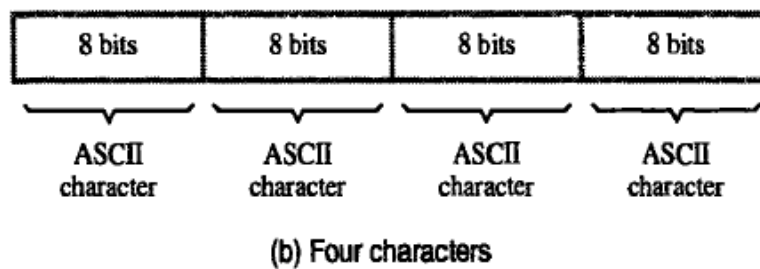
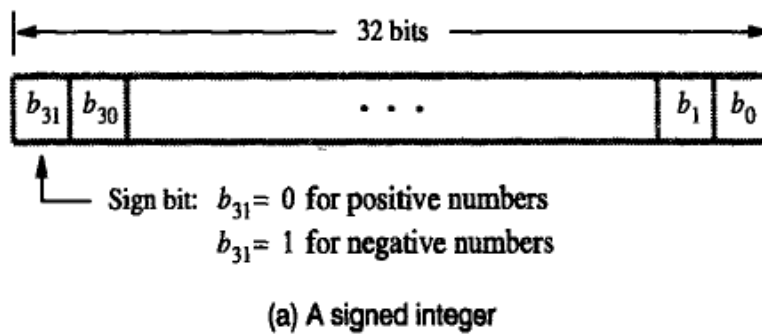


Fig. 1.3 Example of encoded information in 32- bit word

- Byte locations have addresses 0,1,

BIG – ENDIAN AND LITTLE – ENDIAN ASSIGNMENTS

There are two ways that byte addresses can be assigned across words.

- Big-Endian – lower byte addresses are used for the more significant bytes of a word
- Little-Endian – lower byte addresses are used for the less significant bytes of a word

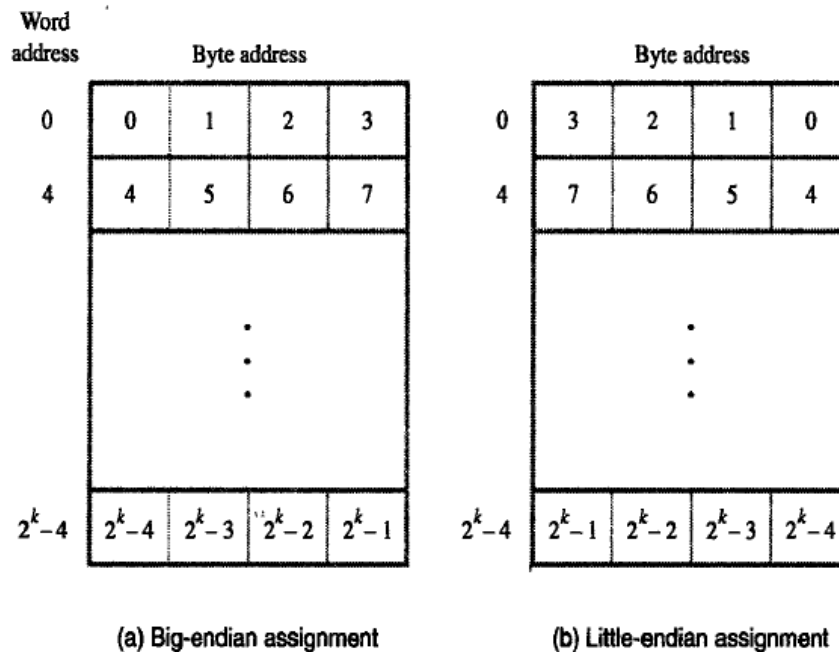


Fig.1.4 Byte and word addressing

WORD ALIGNMENT

In the case of 32-bit word length, natural word boundaries occur at addresses 0, 4, 8, ... as shown in the (fig. byte and word addressing). The word locations have aligned addresses. Words are said to be aligned in memory if they begin at a byte address that is a multiple of the number of bytes in a word.

There is no fundamental reason why words cannot begin at an arbitrary byte address. In that case words are said to have unaligned addresses.

ACCESSING NUMBER, CHARACTERS AND CHARACTER STRINGS

A number usually occupies one word. It can be accessed in the memory by specifying its word address. Similarly individual characters can be accessed by a byte address. The beginning of the string is indicated by giving the address of the byte containing its first character. Successive byte locations contain successive characters of the string.

There are two ways to indicate the length of the string.

3. The special control character with the meaning “end of string” can be used as last character in the string.
4. Processor register can contain a number indicating the length of the string in bytes.

MEMORY OPERATIONS

To execute an instruction, the processor control circuits must cause the word containing the instruction to be transferred from the memory to the processor. Operands and results must also be moved between the memory and the processor. Two basic operations involving the memory are needed, namely load and store.

The load operation transfers a copy of the contents of a specific memory location to the processor. The memory contents remain unchanged. To start a load operation, the processor sends the address of the desired location to the memory and requests that its contents be read. The memory reads the data stored at that address and sends them to the processor.

The store operation transfers an item of information from the processor to a specific memory location, destroying the former contents of that location. The processor sends the address of the desired location to the memory, together with the data to be written into that location.

INSTRUCTION AND INSTRUCTION SEQUENCING

The tasks carried out by a computer program consist of a sequence of small steps, such as adding two numbers, testing for a particular condition, reading a character from the keyboard, or sending a character to be displayed on a display screen.

A computer must have instructions capable of performing four types of operation:

- Data transfers between the memory and the processor registers
- Arithmetic and logic operations on data
- Program sequencing and control
- I/O transfers

REGISTER TRANSFER NOTATION

- Identify a location by a symbolic name standing for its hardware binary address (LOC, R0,...)
- Contents of a location are denoted by placing square brackets around the name of the location ($R1 \leftarrow [LOC]$, $R3 \leftarrow [R1] + [R2]$)
- Register Transfer Notation (RTN)

ASSEMBLY LANGUAGE NOTATION

- Represent machine instructions and programs.
- Move LOC, $R1 = R1 \leftarrow [LOC]$
- Add R1, R2, $R3 = R3 \leftarrow [R1] + [R2]$

BASIC INSTRUCTION TYPES

- Three-Address Instructions
 - ADD R1, R2, R3 $R1 \leftarrow R2 + R3$
- Two-Address Instructions
 - ADD R1, R2 $R1 \leftarrow R1 + R2$
- One-Address Instructions
 - ADD M $AC \leftarrow AC + M[AR]$
- Zero-Address Instructions
 - ADD $TOS \leftarrow TOS + (TOS - 1)$
- RISC Instructions
 - Lots of registers. Memory is restricted to Load & Store

Example: Evaluate $(A+B) * (C+D)$

Three-Address	Two-Address
ADD R1, A, B; $R1 \leftarrow M[A] + M[B]$ ADD R2, C, D; $R2 \leftarrow M[C] + M[D]$ MUL X, R1, R2; $M[X] \leftarrow R1 * R2$	MOV R1, A; $R1 \leftarrow M[A]$ ADD R1, B ; $R1 \leftarrow R1 + M[B]$ MOV R2, C ; $R2 \leftarrow M[C]$ ADD R2, D ; $R2 \leftarrow R2 + M[D]$ MUL R1, R2 ; $R1 \leftarrow R1 * R2$ MOV X, R1 ; $M[X] \leftarrow R1$
One-Address	Zero-Address
LOAD A ; $AC \leftarrow M[A]$ ADD B ; $AC \leftarrow AC + M[B]$ STORE T ; $M[T] \leftarrow AC$ LOAD C ; $AC \leftarrow M[C]$ ADD D ; $AC \leftarrow AC + M[D]$ MUL T ; $AC \leftarrow AC * M[T]$ STORE X ; $M[X] \leftarrow AC$	PUSH A ; $TOS \leftarrow A$ PUSH B ; $TOS \leftarrow B$ ADD ; $TOS \leftarrow (A + B)$ PUSH C ; $TOS \leftarrow C$ PUSH D ; $TOS \leftarrow D$ ADD ; $TOS \leftarrow (C + D)$ MUL ; $TOS \leftarrow (C+D)*(A+B)$ POP X ; $M[X] \leftarrow TOS$
RISC	
LOAD R1, A ; $R1 \leftarrow M[A]$ LOAD R2, B ; $R2 \leftarrow M[B]$ LOAD R3, C ; $R3 \leftarrow M[C]$	

```

LOAD R4, D ; R4 ← M[D]
ADD  R1, R1, R2 ; R1 ← R1 + R2
ADD  R3, R3, R4 ; R3 ← R3 + R4
MUL  R1, R1, R3 ; R1 ← R1 * R3
STORE X, R1 ; M[X] ← R1

```

INSTRUCTION EXECUTION AND STRAIGHT-LINE SEQUENCING

The three instructions of the program are in successive word locations starting at location i .

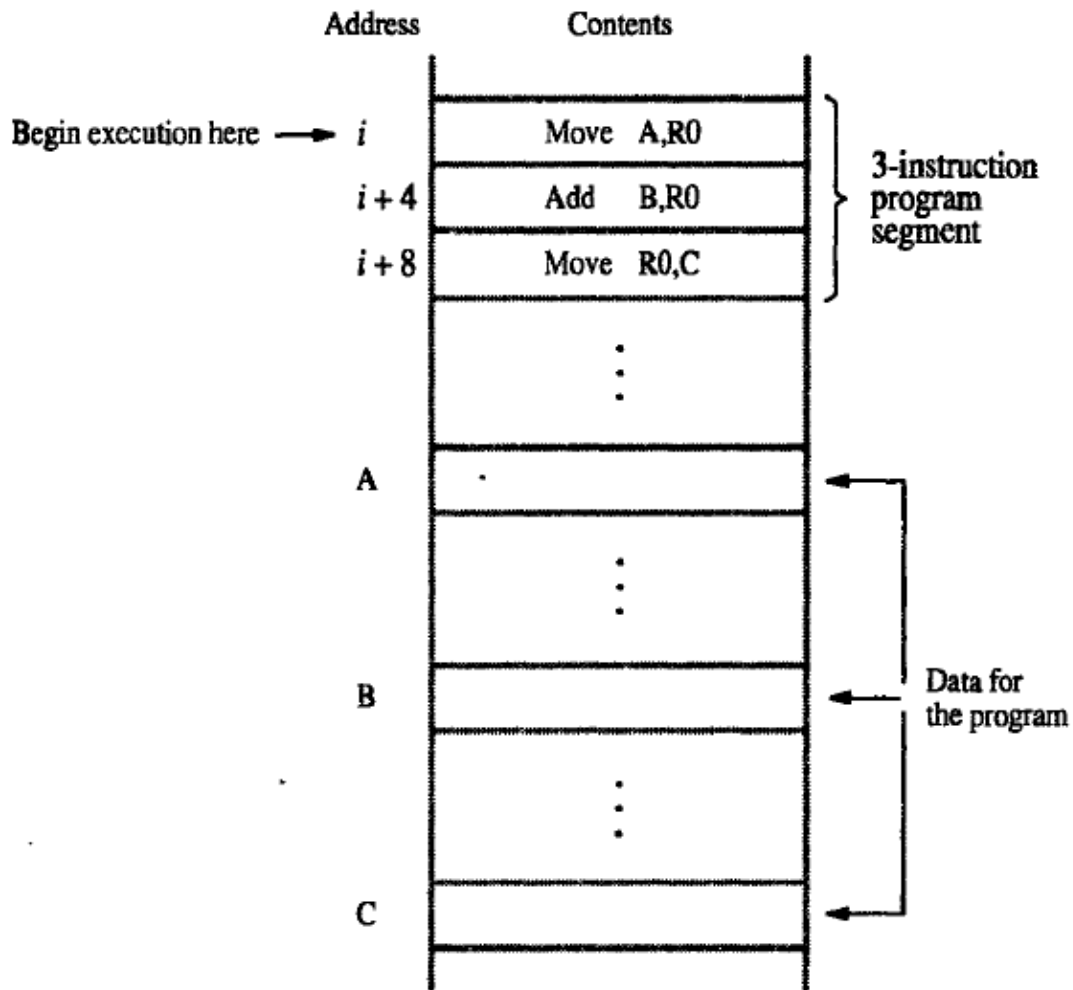


Fig.1.5 A program for $C \leftarrow [a] + [b]$

The processor contains a register called the program counter PC, which the address of the instruction to be executed next.

To begin executing a program the address of its first instruction(i) must be placed into the PC.

The processor control circuits use the information in the PC to fetch and execute instruction, one at the time, in order increasing addresses. This is called straight-line sequencing

Executing a given instruction is a two phase procedure. In the first phase, called instruction fetch, the instruction is fetched from the memory location whose address in the PC. This instruction is placed in the instruction Register(IR) in the processor. The second phase called instruction execute, the instruction in IR is examined to determine which operations is to be performed.

The specified operation is performed by the processor. When the execute phase of an instruction is completed, the PC contains the address of the next instruction, and the new instruction fetch phase can begin.

BRANCHING

- Consider the task of adding a list of n numbers.
- The program outlined in following fig1.6.is a generalization of the program in fig 1.5
- The address of the memory location contain the n numbers are symbolically given as NUM1,NUM2,... NUM n and separate add instruction is used to add each number to the contents of register R0.
- After all the numbers have been added, the result is placed in memory location SUM.

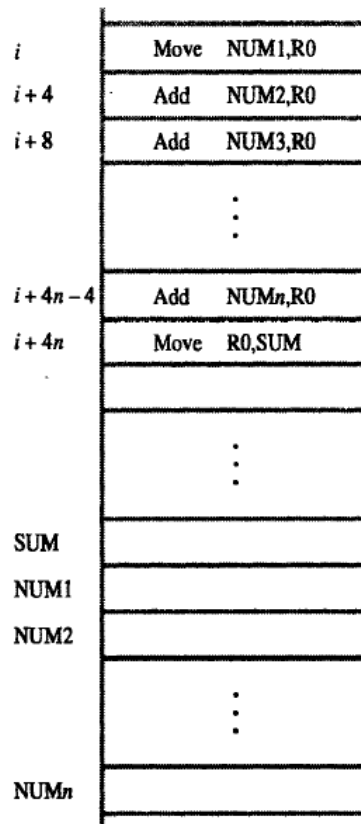


Fig.1.6 A straight line program for adding n numbers

- Instead of using a long list of add instructions, it is possible to place a single ADD instruction in a program loop as shown in Fig. 1.7
- The loop is a straight line sequence of instruction as many times as needed.
- Within the body of the loop, the instruction
Decrement R1
Reduces the contents of R1 by 1 each time through the loop.
- We use branch instruction to load a new value into the program counter.
- As a result a processor fetches and executes the instruction at this new address called the branch target.
- A conditional branch instruction causes a branch only if a specified condition is satisfied.
Branch >0 LOOP

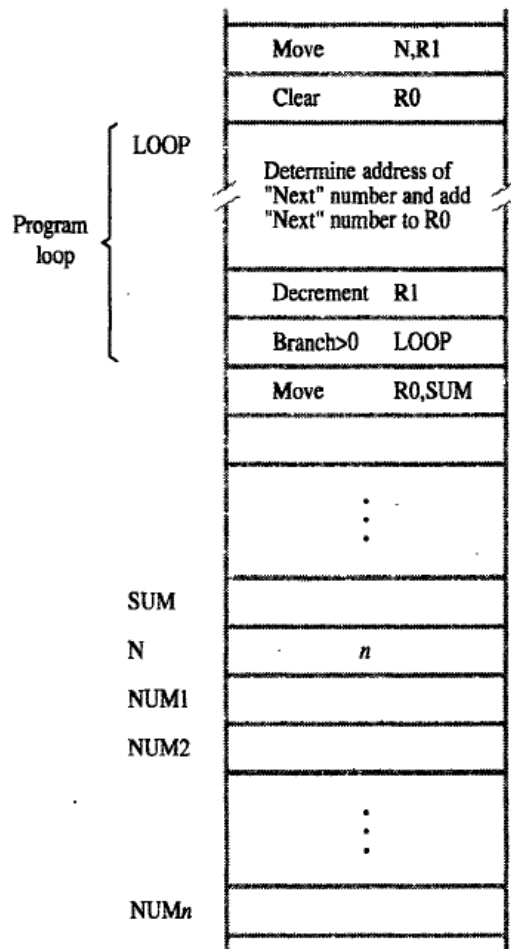


Fig 1.7 Using a loop to add n numbers

CONDITION CODES

The processor keeps track of information about the results of various operations for use by subsequent conditional branch instruction. This is accomplished by recording the required information in individual bits, called condition code flags. These flags are usually grouped together in a special processor register called the condition code register or status register.

Individual condition code flags are set to 1 or cleared to 0, depending on the outcome of the operation perform. Four commonly used flags are

- N (negative)
- Z (zero)
- V (overflow)
- C (carry)

GENERATING MEMORY ADDRESSES

- How to specify the address of branch target?
- Can we give the memory operand address directly in a single Add instruction in the loop?
- Use a register to hold the address of NUM1; then increment by 4 on each pass through the loop.

ADDRESSING MODES

Programmers used organization called data structure to represent the data used in computation. these include list, linked list, queue, arrays. The different ways in which the location of an operand is specified in an instruction are referred to as Addressing modes

- Implied
 - AC is implied in “ADD M[AR]” in “One-Address” instr.
 - TOS is implied in “ADD” in “Zero-Address” instr.
- Immediate
 - The use of a constant in “MOV R1, 5”, i.e. $R1 \leftarrow 5$
- Register
 - Indicate which register holds the operand
- Register Indirect
 - Indicate the register that holds the number of the register that holds the operand
- MOV R1, (R2)
- Autoincrement / Autodecrement
 - Access & update in 1 instr.
- Direct Address
- Use the given address to access a memory location
- Indirect Address
 - Indicate the memory location that holds the address of the memory location that holds the data.
- Indexed
 - $EA = \text{Index Register} + \text{Relative Addr}$
- Base Register
 - $EA = \text{Base Register} + \text{Relative Addr}$
- Relative Address
 - $EA = PC + \text{Relative Addr}$

Name	Assembler syntax	Addressing function
Immediate	#Value	Operand = Value
Register	Ri	EA = Ri
Absolute (Direct)	LOC	EA = LOC
Indirect	(Ri)	EA = [Ri]
	(LOC)	EA = [LOC]
Index	X(Ri)	EA = [Ri] + X
Base with index	(Ri,Rj)	EA = [Ri] + [Rj]
Base with index and offset	X(Ri,Rj)	EA = [Ri] + [Rj] + X
Relative	X(PC)	EA = [PC] + X
Autoincrement	(Ri)+	EA = [Ri]; Increment Ri
Autodecrement	-(Ri)	Decrement Ri; EA = [Ri]

EA = effective address
Value = a signed number

INDEXING AND ARRAYS

- Index mode – the effective address of the operand is generated by adding a constant value to the contents of a register.
- Index register
- $X(R_i)$: $EA = X + [R_i]$
- The constant X may be given either as an explicit number or as a symbolic name representing a numerical value.
- If X is shorter than a word, sign-extension is needed
- In general, the Index mode facilitates access to an operand whose location is defined relative to a reference point within the data structure in which the operand appears.
- Several variations:
 - (R_i, R_j) : $EA = [R_i] + [R_j]$
 - $X(R_i, R_j)$: $EA = X + [R_i] + [R_j]$

RELATIVE ADDRESSING

- Relative mode – the effective address is determined by the Index mode using the program counter in place of the general-purpose register.
- $X(PC)$ – note that X is a signed number

- Branch>0 LOOP
- This location is computed by specifying it as an offset from the current value of PC.
- Branch target may be either before or after the branch instruction, the offset is given as a signed num.

ADDITIONAL MODES

- Autoincrement mode – the effective address of the operand is the contents of a register specified in the instruction. After accessing the operand, the contents of this register are automatically incremented to point to the next item in a list.
- $(R_i)+$. The increment is 1 for byte-sized operands, 2 for 16-bit operands, and 4 for 32-bit operands.
- Autodecrement mode: $-(R_i)$ – decrement first

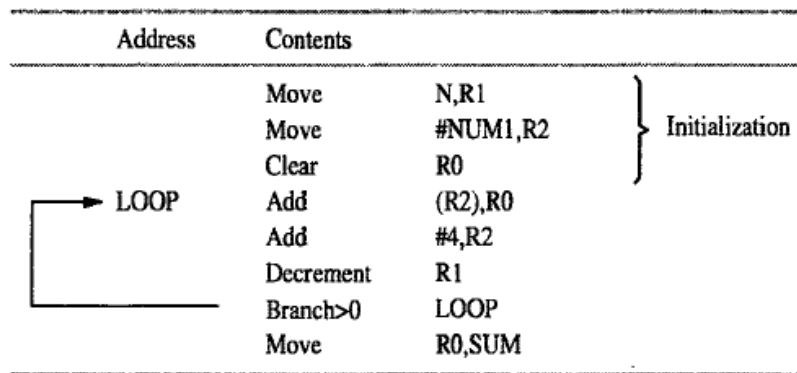


Fig.1.8 The Autoincrement addressing mode

ASSEMBLY LANGUAGE

Machine Instructions Are Represented By patterns of 0s and 1s. symbolic names are used to represent patterns. When writing programs for specific computer, such words are normally replaced by acronyms called mnemonics such as MOV,ADD,INC, and BR. R3 is used to refer register 3, and LOC to refer to a location.

A complete set of such symbolic names and rules for their use constitute a programming language, generally referred to as an assembly language.

The set of rules for using the mnemonics in the specification of complete instructions and programs is called the syntax of the language.

An assembler is a type of computer program that interprets software programs written in assembly language into machine language, code and instructions that can be executed by a computer.

The user program in its original alphanumeric text format is called a source program, and the assembled machine language program is called an object program.

A typical assembly language consists of 3 types of instruction statements that are used to define program operations:

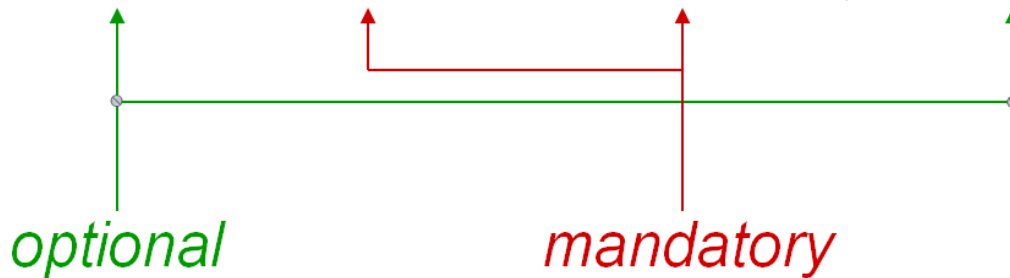
- Opcode mnemonics
- Data definitions
- Assembly directives

ASSEMBLY DIRECTIVES

Pseudo-operations

- do not refer to operations executed by program
- used by assembler
- look like instruction, but “opcode” starts with dot
- Each line of a program is one of the following:
 - an instruction
 - an assembler directive (or pseudo-op)
 - a comment
- Whitespace (between symbols) and case are ignored.
- Comments (beginning with “;”) are also ignored.
- An instruction has the following format:

LABEL OPCODE OPERANDS ; COMMENTS



Opcode	Operand	Meaning
.ORIG	address	starting address of program
.END		end of program
.BLKW	n	allocate n words of storage
.FILL	n	allocate one word, initialize with value n
.STRINGZ	n-character string	allocate n+1 locations, initialize w/characters and null terminator

BASIC INPUT/OUTPUT OPERATIONS

- The data on which the instructions operate are not necessarily already stored in memory.
- Data need to be transferred between processor and outside world (disk, keyboard, etc.)
- I/O operations are essential, the way they are performed can have a significant effect on the performance of the computer.

Program-Controlled I/O

- Read in character input from a keyboard and produce character output on a display screen.

Example

- Rate of data transfer (keyboard, display, processor)
- Difference in speed between processor and I/O device creates the need for mechanisms to synchronize the transfer of data.

- A solution: on output, the processor sends the first character and then waits for a signal from the display that the character has been received. It then sends the second character. Input is sent from the keyboard in a similar way.

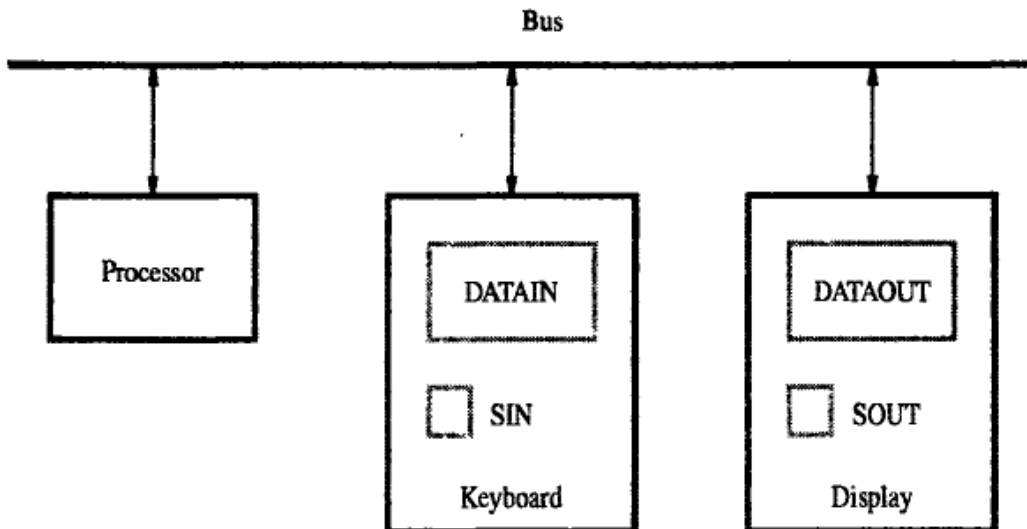


Fig. 1.9 Bus connection for processor, keyboard, and display.

- Machine instructions that can check the state of the status flags and transfer data:

READWAIT Branch to READWAIT if SIN = 0

Input from DATAIN to R1

WRITEWAIT Branch to WRITEWAIT if SOUT = 0

Output from R1 to DATAOUT

- Memory-Mapped I/O – some memory address values are used to refer to peripheral device buffer registers. No special instructions are needed. Also use device status registers.

READWAIT Testbit #3, INSTATUS

Branch=0 READWAIT

MoveByte DATAIN, R1

- Assumption – the initial state of SIN is 0 and the initial state of SOUT is 1.
- Any drawback of this mechanism in terms of efficiency?
- Two wait loops → processor execution time is wasted

STACK

In order to organize the control and information linkage between the main program and the subroutine, the datastructure called the stack is used. A stack is a list of data elements, usually words or bits, with the accessing restrictions that elements can be added or removed at one end of the list only. This end is called the top of the stack and other end is called the bottom. The structure is sometimes referred to as Push down stack. It uses LIFO principle for two operations-Push and Pop.

The processor register is used to keep track of the address of the element of the stack i.e at the top at any given time. This register is called stack pointer(SP).

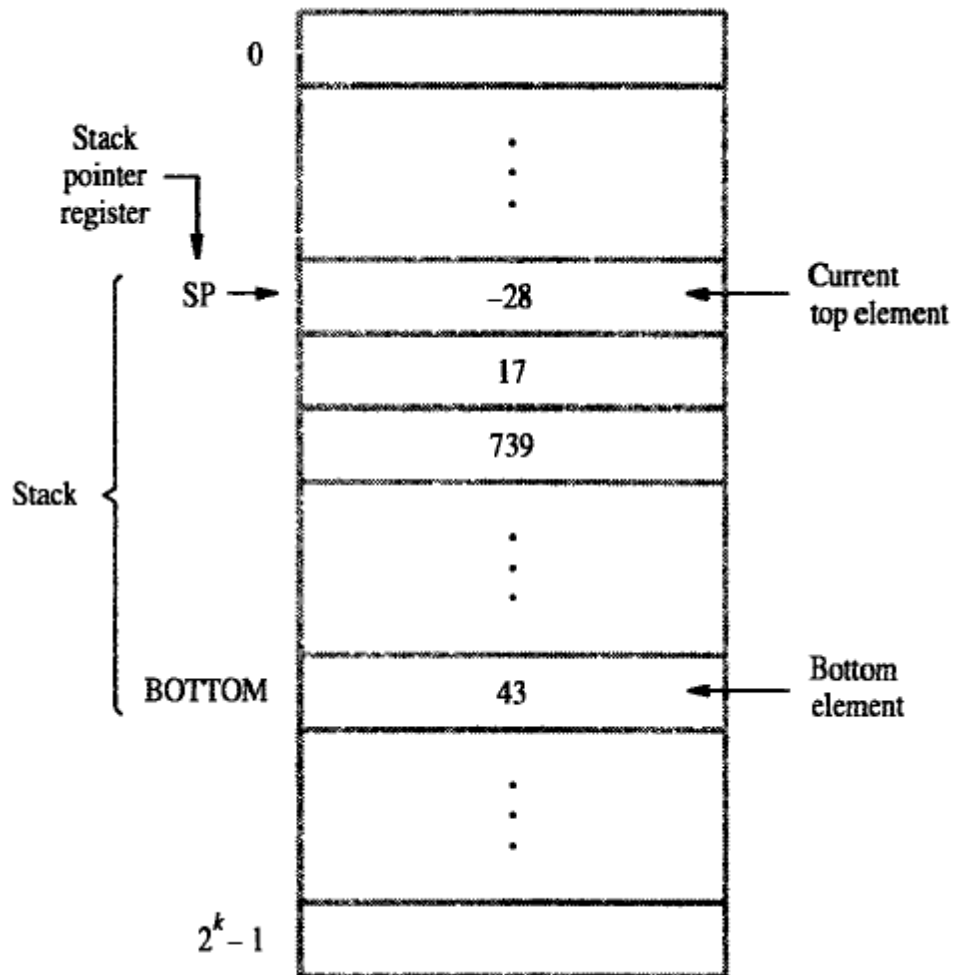


Fig. 1.10 A stack of words in the memory

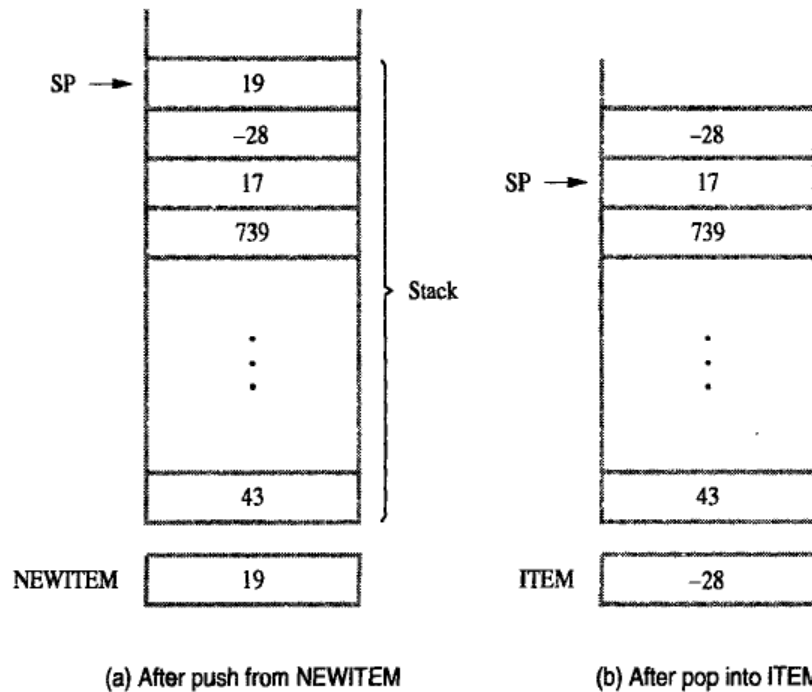


Fig 1.11 Effect of stack operation on the stack in the Fig 1.10

SAFEPOP	Compare	#2000,SP	Check to see if the stack pointer contains an address value greater than 2000. If it does, the stack is empty. Branch to the routine EMPTYERROR for appropriate action.
	Branch>0	EMPTYERROR	
	Move	(SP)+,ITEM	Otherwise, pop the top of the stack into memory location ITEM.

(a) Routine for a safe pop operation

SAFEPUSH	Compare	#1500,SP	Check to see if the stack pointer contains an address value equal to or less than 1500. If it does, the stack is full. Branch to the routine FULLERROR for appropriate action.
	Branch≤0	FULLERROR	
	Move	NEWITEM,-(SP)	Otherwise, push the element in memory location NEWITEM onto the stack.

(b) Routine for a safe push operation

Fig 1.12 Checking for empty and full errors in POP and PUSH operations.

QUEUE

Another useful datastructure that is similar to stack is called the queue. Data are stored and retrieved from a queue on FIFO basics. New data are added at the back and retrieved from the front.

There are 2 important differences between how a stack and a queue is implemented.

One end of the is fixed while the other end rises and falls as data are pushed and popped. A single pointer is needed to point to top of the stack, at on the other hand both ends of the queue move to higher addresses as data are added at the back and removed from the front. So two pointers are needed to keep track of the two ends of the queue.

One way to limit the queue to fixed region in a memory is to use a circular buffer.

SUBROUTINES

In a given program it is often necessary to perform a particular sub-task. Many times on different data values ,such a sub-task is usually called as sub-routines.

When a program branches to a sub-routines we say that it is calling the sub-routine. The instruction that performs this branch operation is named a call instruction.

After a sub-routine as been executed a calling program must resume execution, continuing immediately after the instruction that called the sub-routine. The sub-routine is said to return the program that called it by executing the return instruction.

Subroutine linkage – The way in which a computer makes it possible to and return from subroutine.

Link register – A register dedicated to the simplest subroutine linkage method is to save the return address in a specific location.

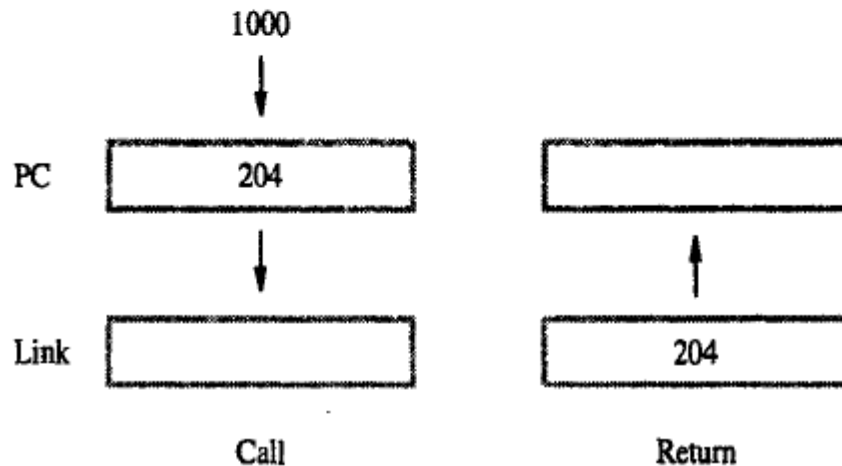
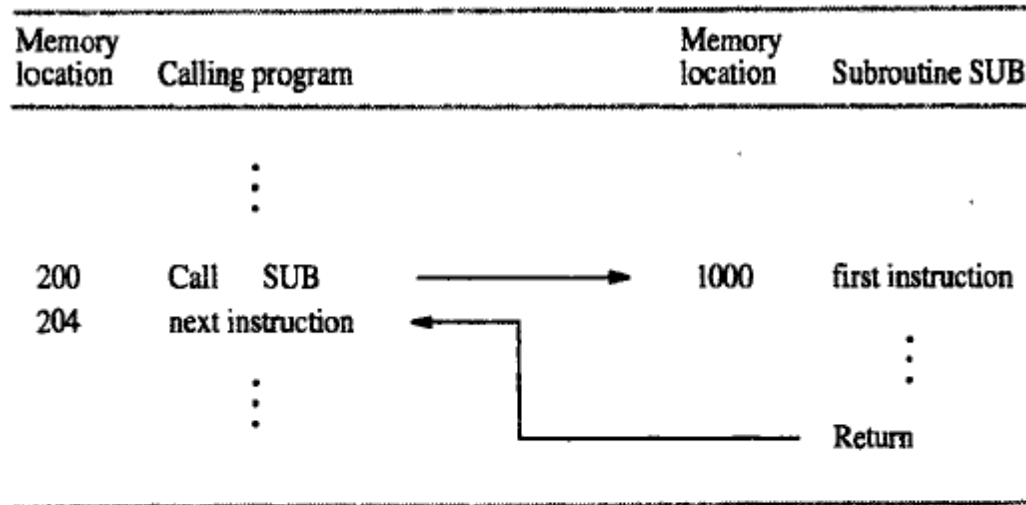


Fig 1.13 Subroutine linkage using A link register

SUBROUTINE NESTING

Subroutine nesting is to have one subroutine call another. In this case, the return address of the second call is also stored in the link register, destroying its previous contents. Hence, it is essential to save the contents of the link register in some other location before calling another subroutines. Otherwise, the return address of the first subroutine will be lost.

THE PROCESSOR STACK

The stack pointer points to the stack called the processor stack. The call instruction pushes the contents of the PC on to the processor stack and loads the subroutine address into the PC. The return instruction pops the return address from the processor stack into the PC.

PARAMETER PASSING

The exchange of information between a calling program and a subroutine is referred to as parameter passing. Parameter passing may be accomplished in several ways passing parameter through processor register is straight forward and efficient. Fig 1.14 shows how the program in fig 1.8 for adding a list of numbers can be implemented as a subroutine, with the parameters passed through registers.

Calling program

Move	N,R1	R1 serves as a counter.
Move	#NUM1,R2	R2 points to the list.
Call	LISTADD	Call subroutine.
Move	R0,SUM	Save result.
:		
:		

Subroutine

LISTADD	Clear	R0	Initialize sum to 0.
LOOP	Add	(R2)+,R0	Add entry from list.
	Decrement	R1	
	Branch>0	LOOP	
	Return		Return to calling program.

Fig 1.14 program of the Fig.1.8 written as a subroutine; parameters passed through registers

Parameter passing by values and by reference

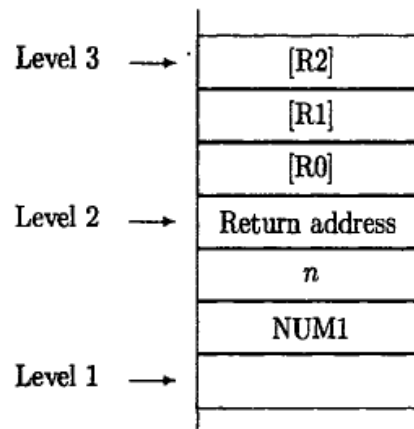
Note the nature of two parameters, NUM1 and n, passed to the subroutines in fig1.14 and fig 1.15 The purpose of the subroutine is to add a list of numbers. Instead of passing the actual list entries, the calling program passes the address of the first number in the list. This technique is passing by reference.

The second parameter is passed by value ,i.e the actual number of entries, n, is passed to the subroutine

Assume top of stack is at level 1 below.

	Move	#NUM1, -(SP)	Push parameters onto stack.
	Move	N, -(SP)	
	Call	LISTADD	Call subroutine (top of stack at level 2).
	Move	4(SP), SUM	Save result.
	Add	#8, SP	Restore top of stack (top of stack at level 1).
	:		
LISTADD	MoveMultiple	R0-R2, -(SP)	Save registers (top of stack at level 3).
	Move	16(SP), R1	Initialize counter to n .
	Move	20(SP), R2	Initialize pointer to the list.
	Clear	R0	Initialize sum to 0.
LOOP	Add	(R2)+, R0	Add entry from list.
	Decrement	R1	
	Branch>0	LOOP	
	Move	R0, 20(SP)	Put result on the stack.
	MoveMultiple	(SP)+, R0-R2	Restore registers.
	Return		Return to calling program.

(a) Calling program and subroutine



(b) Top of stack at various times

Fig 1.15 program of fig 1.8 written as a subroutine, parameters passed on the stack.

THE STACK FRAME

During execution of the subroutine, six locations at the top of the stack contain entries that are needed by the subroutines. These locations constitute a private workspace for the subroutine, created at the time the subroutine is entered and freed up when the subroutines returns control to the calling program. Such space is called a stack frame.

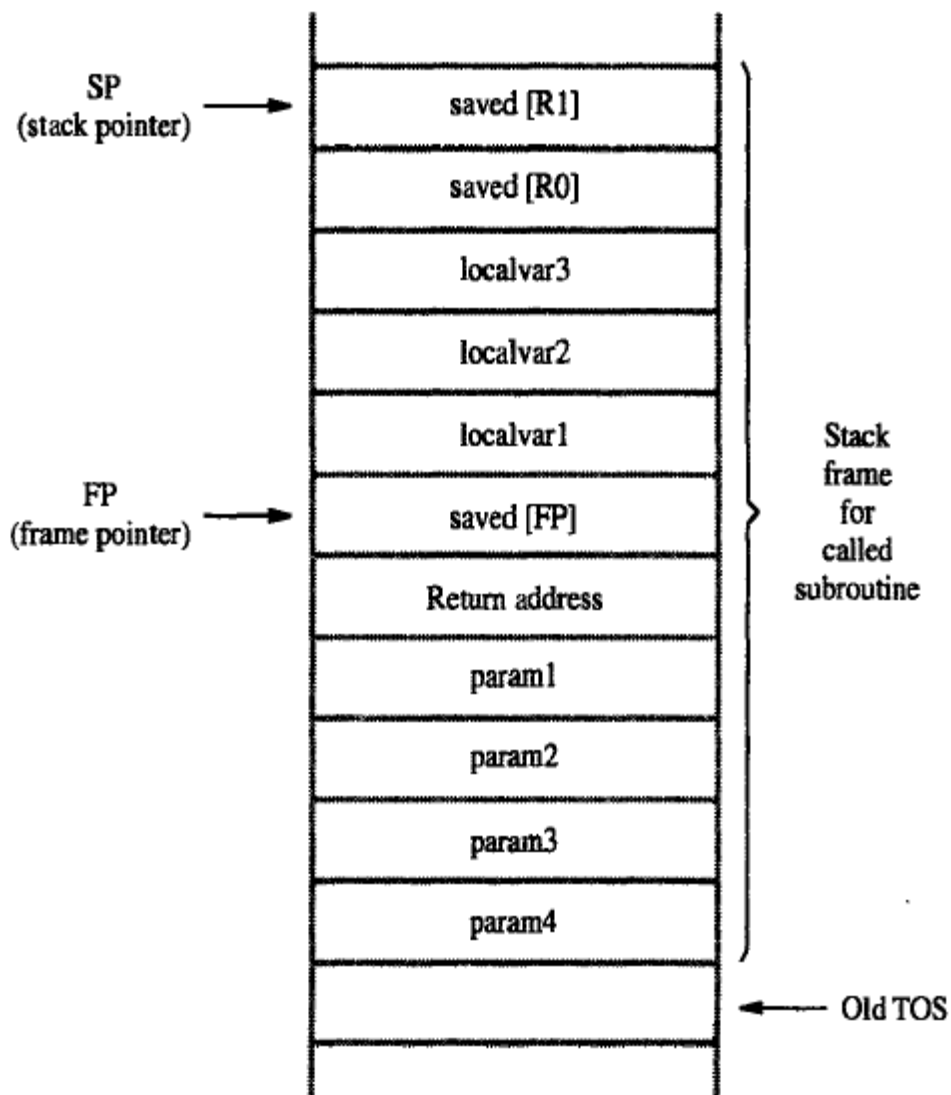


Fig 1.16 A subroutines stack frame example.

Fig 1.16 shows a example of a commonly used layout for information in a stack frame. In addition to a stack pointer(SP), it is useful to have another pointer register called the frame pointer(FP), for convenient access to the parameter passed to the subroutine and to the local memory variable used by the subroutines.

Memory location	Instructions		Comments	
Main program				
	:			
2000	Move	PARAM2, -(SP)	Place parameters on stack.	
2004	Move	PARAM1, -(SP)		
2008	Call	SUB1		
2012	Move	(SP), RESULT	Store result.	
2016	Add	#8, SP	Restore stack level.	
2020	next instruction			
	:			
First subroutine				
2100	SUB1	Move	FP, -(SP)	Save frame pointer register.
2104		Move	SP, FP	Load the frame pointer.
2108		MoveMultiple	R0-R3, -(SP)	Save registers.
2112		Move	8(FP), R0	Get first parameter.
		Move	12(FP), R1	Get second parameter.
		:		
		Move	PARAM3, -(SP)	Place a parameter on stack.
2160		Call	SUB2	
2164		Move	(SP)+, R2	Pop SUB2 result into R2.
		:		
		Move	R3, 8(FP)	Place answer on stack.
		MoveMultiple	(SP)+, R0-R3	Restore registers.
		Move	(SP)+, FP	Restore frame pointer register.
		Return		Return to Main program.
Second subroutine				
3000	SUB2	Move	FP, -(SP)	Save frame pointer register.
		Move	SP, FP	Load the frame pointer.
		MoveMultiple	R0-R1, -(SP)	Save registers R0 and R1.
		Move	8(FP), R0	Get the parameter.
		:		
		Move	R1, 8(FP)	Place SUB2 result on stack.
		MoveMultiple	(SP)+, R0-R1	Restore registers R0 and R1.
		Move	(SP)+, FP	Restore frame pointer register.
		Return		Return to Subroutine 1.

Fig 1.17 A nested subroutine

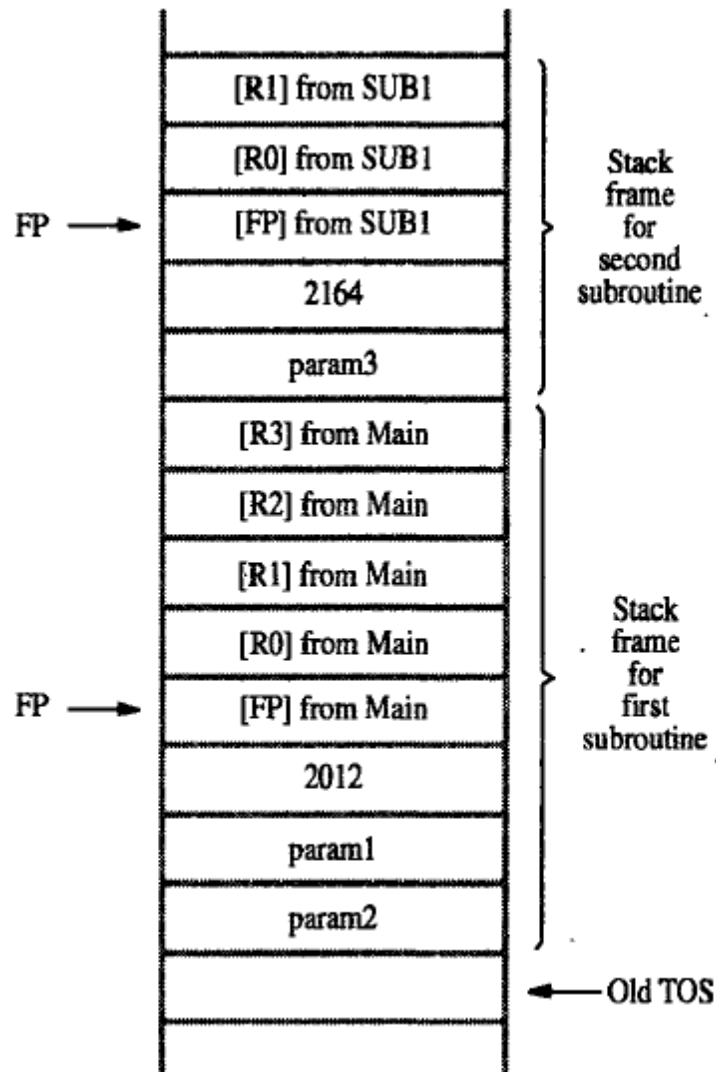


Fig 1.18 Stack frames fig 1.17

SHIFT AND ROTATE INSTRUCTIONS

There are many applications that require the bits of an operand to be shifted right or left some specified number of bit positions.

The details of how the shifts are performed depend on whether the operand is a signed number or some more general binary coded information. For general operand we use a logical shift, for number we use an arithmetic shift which preserves the sign of the number.

Logical Shift:

Two logical shift instructions are needed, one for shifting left (LShiftL) and another for shifting right (LShiftR).

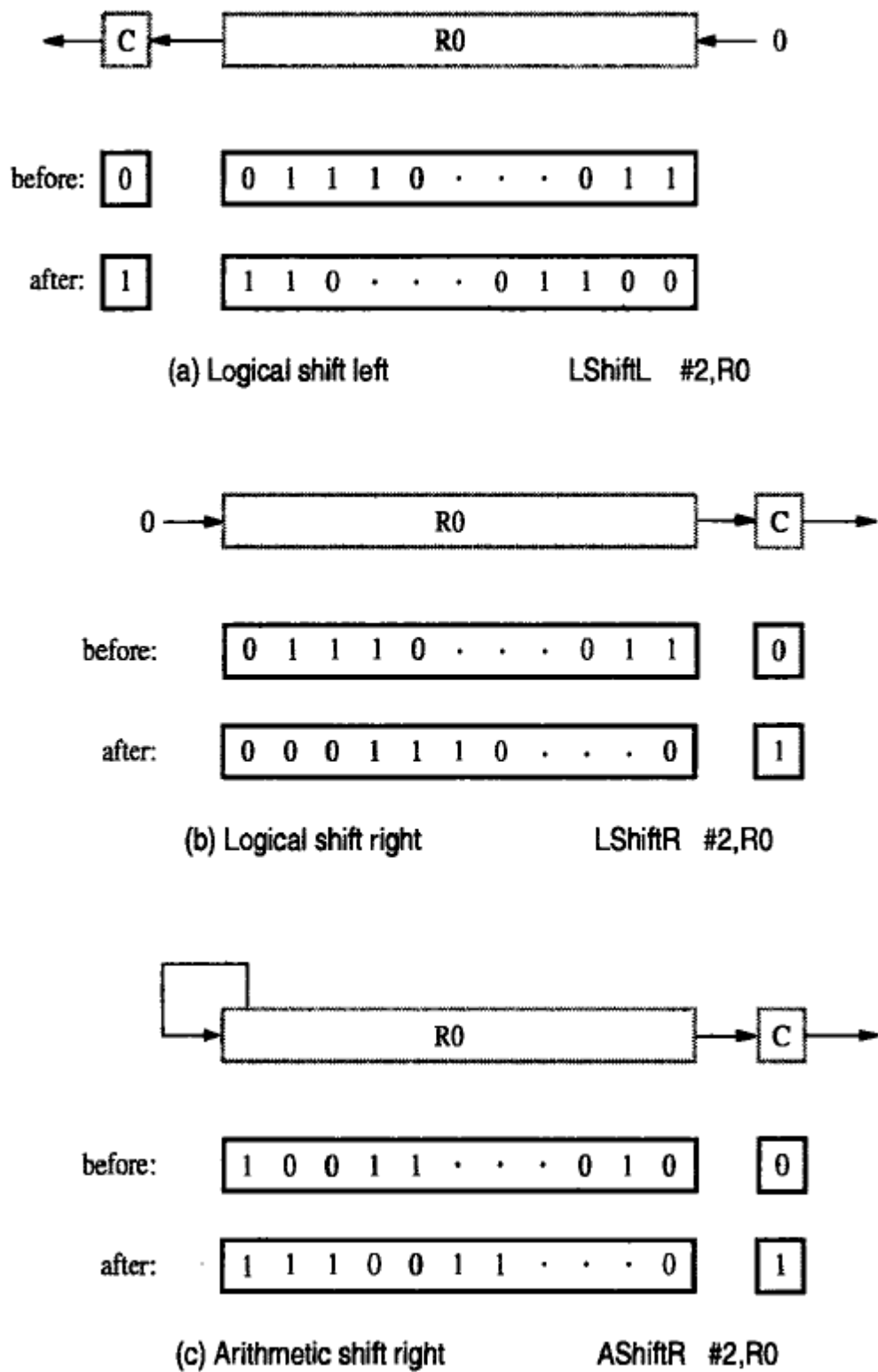


Fig 1.19 Logical and arithmetic shift instruction

Move	#LOC,R0	R0 points to data.
MoveByte	(R0)+,R1	Load first byte into R1.
LShiftL	#4,R1	Shift left by 4 bit positions.
MoveByte	(R0),R2	Load second byte into R2.
And	#\$F,R2	Eliminate high-order bits.
Or	R1,R2	Concatenate the BCD digits.
MoveByte	R2,PACKED	Store the result.

Fig 1.20 A routine that packs two BCD digits

Arithmetic Shifts

- Shifts may occur right and left.
- The requirement on right shifting distinguishes arithmetic shifts from logical in which the fill- in bits is always zero.
- Right shift – AShiftR
- Left shift – AShiftL
- The arithmetic left shift is exactly same as the logical left shift.

Rotate operation

- In the shift operations, the bit shifted out of the operand are lost, except for the last bit shifted out which is retain in the carry flag C.
- To preserve all bits, a set of rotate instructions can be used.
- Two versions of both the left and the right rotate instruction are usually provided.
- The mnemonics Rotate L, Rotate LC, Rotate R, and Rotate Rc, denote the instruction that the rotate operations.

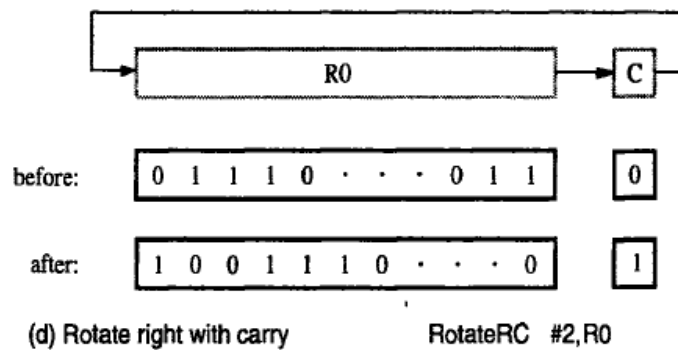
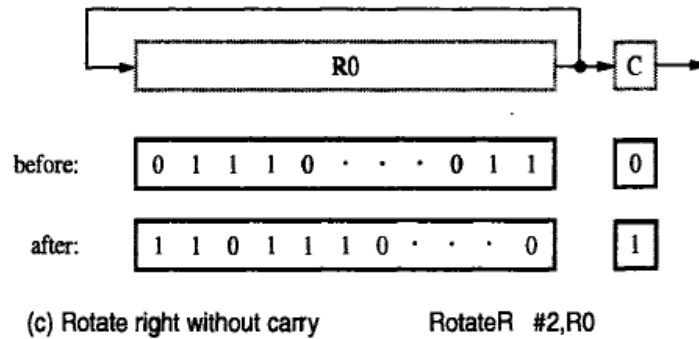
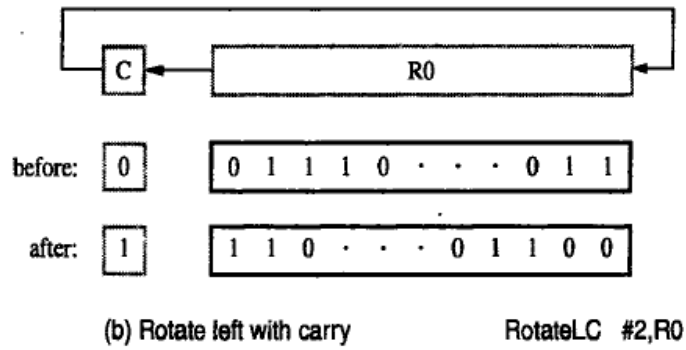
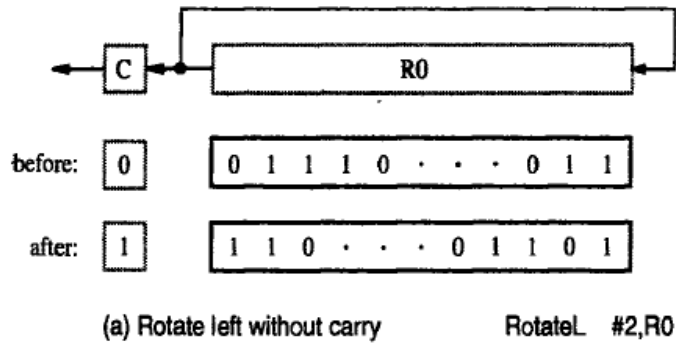


Fig1.21 Rotate instructions

BYTE-SORTING PROGRAM

```

for (j = n-1; j > 0; j = j - 1)
{
    for (k = j-1; k >= 0; k = k - 1)
    {
        if (LIST[k] > LIST[j])
        {
            TEMP = LIST[k];
            LIST[k] = LIST[j];
            LIST[j] = TEMP;
        }
    }
}

```

(a) C-language program for sorting

	Move	#LIST,R0	Load LIST into base register R0.
	Move	N,R1	Initialize outer loop index
	Subtract	#1,R1	register R1 to $j = n - 1$.
OUTER	Move	R1,R2	Initialize inner loop index
	Subtract	#1,R1	register R2 to $k = j - 1$.
	MoveByte	(R0,R1),R3	Load LIST(j) into R3, which holds current maximum in sublist.
INNER	CompareByte	R3,(R0,R2)	If LIST(k) $\leq$ [R3],
	Branch ≤ 0	NEXT	do not exchange.
	MoveByte	(R0,R2),R4	Otherwise, exchange LIST(k)
	MoveByte	R3,(R0,R2)	with LIST(j) and load
	MoveByte	R4,(R0,R1)	new maximum into R3.
	MoveByte	R4,R3	Register R4 serves as TEMP.
NEXT	Decrement	R2	Decrement index registers R2 and
	Branch ≥ 0	INNER	R1, which also serve as
	Decrement	R1	as loop counters, and branch
	Branch > 0	OUTER	back if loops not finished.

(b) Assembly language program for sorting

Fig 1.22 A byte-sorting program using straight – selection sort.